

AI Assisted Coding

Assignment - 6

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Code:

```
lab6.py > ...
1  # assume you are a developer working on creating a student information module.
2  # Your task is to implement the following functionality:
3  # create a student class with attributes
4  # name:String
5  # rollNumber: unique integer
6  # branch : string
7  # create a method to display_details to print student information in a proper manner
8  # so i want the expected output
9  # A py class with a __init__ and display_details method and aslo a sample obj creation and
10 # output displayed on console
11 class Student:
12     def __init__(self, name, roll_number, branch):
13         self.name = name # Initialize the name attribute
14         self.roll_number = roll_number # Initialize the roll number attribute
15         self.branch = branch # Initialize the branch attribute
16     def display_details(self):
17         """
18         Display the details of the student in a proper manner.
19         """
20         print(f"Name: {self.name}") # Print the student's name
21         print(f"Roll Number: {self.roll_number}") # Print the student's roll number
22         print(f"Branch: {self.branch}") # Print the student's branch
23 # Sample object creation and output display
24 if __name__ == "__main__":
25     student1 = Student("Alice", 101, "Computer Science") # Create a sample student object
26     student1.display_details() # Call the method to display the student's details
27     # Expected Output:
28     # Name: Alice
29     # Roll Number: 101
30     # Branch: Computer Science
31
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python311/Python.exe" -i -c "import sys; sys.path.append('C:/Users/NITHIN REDDY/Desktop/All courses/AI assistant/lab6.py'); from lab6 import Student; student1 = Student('Alice', 101, 'Computer Science'); student1.display_details()"
Name: Alice
Roll Number: 101
Branch: Computer Science
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number

using a loop.

- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Code:

```
lab6.py > multiples_while_loop
33 # Assume you are a developer developing a mode to make the multiples of a number by using
34 # python so your task it to:
35 # create a function that takes an interger from user and print the multiples of that number upto
36 # 10 multiples using a loop
37 # for better understanding i want to use the following program with 2 types of loops for loop
38 # and while loop and compare the both code
39 def multiples_for_loop(number):
40     """
41     Print the multiples of a number up to 10 multiples using a for loop.
42
43     Parameters:
44     number (int): The number for which to print the multiples.
45     """
46     print(f"Multiples of {number} using for loop:")
47     for i in range(1, 11): # Loop from 1 to 10
48         print(number * i, end=" ") # Print the multiple of the number
49 def multiples_while_loop(number):
50     """
51     Print the multiples of a number up to 10 multiples using a while loop.
52
53     Parameters:
54     number (int): The number for which to print the multiples.
55     """
56     print(f"\n Multiples of {number} using while loop:")
57     i = 1 # Initialize the counter
58     while i <= 10: # Loop until the counter is less than or equal to 10
59         print(number * i, end=" ") # Print the multiple of the number
60         i += 1 # Increment the counter
61 if __name__ == "__main__":
62     num = int(input("Enter a number to find its multiples: ")) # Take user input
63     multiples_for_loop(num) # Call the function to print multiples using for loop
64     multiples_while_loop(num) # Call the function to print multiples using while loop
65     # Comparison:
66     # Both functions achieve the same result of printing the multiples of the given number up to 10 multiples.
67     # The for loop is generally more concise and easier to read when the number of iterations is
68     # known beforehand, while the while loop can be more flexible for cases where the number of iterations is not known.
69
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python
istant/lab6.py"
Enter a number to find its multiples: 5
Multiples of 5 using for loop:
5 10 15 20 25 30 35 40 45 50
Multiples of 5 using while loop:
5 10 15 20 25 30 35 40 45 50
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups

(e.g., child, teenager, adult, senior).

- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Code:

```
117
118 # assume you are building a simple python module to classify the ages of people into different categories based on their age.
119 # using conditionals statements we need to classify the ages into child, teenager, adult, and senior citizen.
120 # so your task is to implement the following functionality:
121 # create a function that takes an integer input from user representing age and classify it into the following
122 # categories:
123 # child: 0-12 years
124 # teenager: 13-19 years
125 # adult: 20-59 years
126 # senior citizen: 60 years and above
127 def classify_age(age):
128     """
129     Classify the age into different categories.
130
131     Parameters:
132     age (int): The age to classify.
133
134     Returns:
135     str: The category of the age.
136     """
137     if age < 0:
138         return "Invalid age" # Handle negative ages
139     elif age <= 12:
140         return "Child" # Age between 0 and 12
141     elif age <= 19:
142         return "Teenager" # Age between 13 and 19
143     elif age <= 59:
144         return "Adult" # Age between 20 and 59
145     else:
146         return "Senior Citizen" # Age 60 and above
147 if __name__ == "__main__":
148     age_input = int(input("Enter the age to classify: ")) # Take user input for age
149     category = classify_age(age_input) # Classify the age using the function
150     print(f"The age {age_input} is classified as: {category}") # Print the classification result
151     # Expected Output:
152     # If the user enters 10, the output will be: The age 10 is classified as: Child
153     # If the user enters 15, the output will be: The age 15 is classified as: Teenager
154     # If the user enters 30, the output will be: The age 30 is classified as: Adult
155     # If the user enters 65, the output will be: The age 65 is classified as: Senior Citizen
```

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe" C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant\lab6.py
Enter the age to classify: 55
The age 55 is classified as: Adult
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> |
```

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Code:

```
72 # Assume you are a developer working on creating a module to compare for loop and while
73 # loop in python
74 # so your task is to implement the following functionality:
75 # create a function that takes an integer from user
76 # you need to calculate the 1st sum of n natural numbers using for loop and while loop both
77 # and compare the both code
78 # First i want to execute the code using for loop and then using while loop and then compare
79 # the both code
80 def sum_natural_numbers_for_loop(n):
81     """
82     Calculate the sum of the first n natural numbers using a for loop.
83     Parameters:
84     n (int): The number of natural numbers to sum.
85     Returns:
86     int: The sum of the first n natural numbers.
87     """
88     total_sum = 0 # Initialize the sum variable
89     for i in range(1, n + 1): # Loop from 1 to n
90         total_sum += i # Add the current number to the total sum
91     return total_sum # Return the calculated sum
92 def sum_natural_numbers_while_loop(n):
93     """
94     Calculate the sum of the first n natural numbers using a while loop.
95     Parameters:
96     n (int): The number of natural numbers to sum.
97     Returns:
98     int: The sum of the first n natural numbers.
99     """
100    total_sum = 0 # Initialize the sum variable
101    i = 1 # Initialize the counter
102    while i <= n: # Loop until the counter is less than or equal to n
103        total_sum += i # Add the current number to the total sum
104        i += 1 # Increment the counter
105    return total_sum # Return the calculated sum
106 if __name__ == "__main__":
107     n = int(input("Enter a number to calculate the sum of the first n natural numbers: ")) # Take user input
108     sum_for_loop = sum_natural_numbers_for_loop(n) # Calculate sum using for loop
109     sum_while_loop = sum_natural_numbers_while_loop(n) # Calculate sum using while loop
110     print(f"Sum of the first {n} natural numbers using for loop: {sum_for_loop}") # Print result from for loop
111     print(f"Sum of the first {n} natural numbers using while loop: {sum_while_loop}") # Print result from while loop
112     # Comparison:
113     # Both functions calculate the same result, which is the sum of the first n natural numbers.
114     # The for loop is more concise and easier to read when the number of iterations is known,
115     # while the while loop can be more flexible for cases where the number of iterations is not predetermined.
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
● PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Pr
istant/lab6.py"
Enter a number to calculate the sum of the first n natural numbers: 145
Sum of the first 145 natural numbers using for loop: 10585
Sum of the first 145 natural numbers using while loop: 10585
○ PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> |
```

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Code:

```
158 # Act as a developer creating a module that you are developing a banking application.
159 # Your task is to implement the following functionality:
160 # create a basic function that such as deposit() withdraw() and check balance().
161 # also add the step by step explanation of the code. and read the input from user and display the output on console.
162 class BankAccount:
163     def __init__(self, account_holder, initial_balance=0):
164         self.account_holder = account_holder # Initialize the account holder's name
165         self.balance = initial_balance # Initialize the account balance
166     def deposit(self, amount):
167         """
168         Deposit a certain amount into the bank account.
169
170         Parameters:
171         amount (float): The amount to be deposited.
172         """
173         if amount > 0: # Check if the deposit amount is positive
174             self.balance += amount # Add the deposit amount to the balance
175             print(f'Deposited ${amount:.2f}. New balance: ${self.balance:.2f}') # Print the new balance
176         else:
177             print("Deposit amount must be positive.") # Handle invalid deposit amounts
178     def withdraw(self, amount):
179         """
180         Withdraw a certain amount from the bank account.
181
182         Parameters:
183         amount (float): The amount to be withdrawn.
184         """
185         if amount > self.balance: # Check if there are sufficient funds for withdrawal
186             print("Insufficient funds.") # Handle insufficient funds
187         elif amount <= 0: # Check if the withdrawal amount is positive
188             print("Withdrawal amount must be positive.") # Handle invalid withdrawal amounts
189         else:
190             self.balance -= amount # Subtract the withdrawal amount from the balance
191             print(f'Withdrew ${amount:.2f}. New balance: ${self.balance:.2f}') # Print the new balance
192     def check_balance(self):
193         """
194         Check and display the current balance of the bank account.
195         """
196         print(f'Current balance: ${self.balance:.2f}') # Print the current balance
```

```

196 print(f"Current balance: ${self.balance:.2f}") # Print the current balance
197 if __name__ == "__main__":
198     account_holder_name = input("Enter the account holder's name: ") # Take user input for account holder's name
199     initial_deposit = float(input("Enter the initial deposit amount: ")) # Take user input for initial deposit
200     account = BankAccount(account_holder_name, initial_deposit) # Create a bank account object with the provided detail
201     while True: # Start an infinite loop to allow multiple transactions
202         print("\nChoose an option:")
203         print("1. Deposit")
204         print("2. Withdraw")
205         print("3. Check Balance")
206         print("4. Exit")
207         choice = input("Enter your choice (1-4): ") # Take user input for the desired action
208         if choice == '1':
209             amount = float(input("Enter the amount to deposit: ")) # Take user input for deposit amount
210             account.deposit(amount) # Call the deposit method
211         elif choice == '2':
212             amount = float(input("Enter the amount to withdraw: ")) # Take user input for withdrawal amount
213             account.withdraw(amount) # Call the withdraw method
214         elif choice == '3':
215             account.check_balance() # Call the check balance method
216         elif choice == '4':
217             print("Thank you for using our banking application. Goodbye!") # Exit message
218             break # Exit the loop and end the program
219         else:
220             print("Invalid choice. Please enter a number between 1 and 4.") # Handle invalid menu choices
221
222

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
● PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe" C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant\lab6.py
Enter the account holder's name: Nithin
Enter the initial deposit amount: 25000

Choose an option:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 2
Enter the amount to withdraw: 1200
Withdrew $1200.00. New balance: $23800.00

Choose an option:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 3
Current balance: $23800.00

Choose an option:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 4
Thank you for using our banking application. Goodbye!
● PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>

```